🚀 Day 21 – Hashing Problems (DSA) Part-2

🚀 1. Majority Element

📌 Problem

Array me wo element find karo jo **n/2 se zyada baar** aata ho.

💡 Approach 1 (HashMap)

- Frequency count karo
- Jo > n/2 ho → answer

💡 Approach 2 (Boyer-Moore Voting 🔥 BEST)

- Ek candidate aur count maintain karo

✅ Code (Optimal)

```
public class MajorityElement {
    public static int majorityElement(int[] nums){
        int count = 0;
        int candidate = 0;

        for(int num : nums){
            if(count == 0){
                candidate = num;
            }

            if(num == candidate){
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }
}
```

⏱️ Complexity

- Time: O(n)
- Space: O(1)

🚀 2. Intersection of Two Arrays

📌 Problem

Do arrays ka common elements find karo.

💡 Approach (HashSet)

- Ek array ko set me daalo
- Dusre array me check karo

✅ Code

```
import java.util.*;

public class Intersection {
    public static int[] intersection(int[] nums1, int[] nums2){
        HashSet<Integer> set1 = new HashSet<>();
        HashSet<Integer> result = new HashSet<>();

        for(int num : nums1){
            set1.add(num);
        }

        for(int num : nums2){
            if(set1.contains(num)){
                result.add(num);
            }
        }

        int[] res = new int[result.size()];
        int i = 0;

        for(int num : result){
            res[i++] = num;
        }

        return res;
    }
}
```

🚀 3. Longest Substring Without Repeating Characters 🔥

📌 Problem

String me longest substring find karo jisme **duplicate na ho**

💡 Approach (Sliding Window + HashSet)

- Window expand karo
- Duplicate mile → shrink karo

✅ Code

```
import java.util.*;

public class LongestSubstring {
    public static int lengthOfLongestSubstring(String s){
        HashSet<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;

        for(int right = 0; right < s.length(); right++){
            while(set.contains(s.charAt(right))){
                set.remove(s.charAt(left));
                left++;
            }

            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }

        return maxLength;
    }
}
```

⏱️ Complexity

- Time: O(n)

🚀 4. Group Anagrams 🔥 🔥

📌 Problem

Same anagram wale strings ko group karo.

💡 Approach

- String sort karo → key bana lo
- Map me group karo

✅ Code

```
import java.util.*;

public class GroupAnagrams {
    public static List<List<String>> groupAnagrams(String[] strs){
        HashMap<String, List<String>> map = new HashMap<>();

        for(String str : strs){
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);

            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }

        return new ArrayList<>(map.values());
    }
}
```

🚀 5. Top K Frequent Elements 🔥 🔥

📌 Problem

Top k most frequent elements return karo.

💡 Approach

1. Frequency count (HashMap)
2. Max Heap use karo

✅ Code

```
import java.util.*;

public class TopKFrequent {
    public static int[] topKFrequent(int[] nums, int k){
        HashMap<Integer, Integer> map = new HashMap<>();

        for(int num : nums){
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        PriorityQueue<Integer> pq = new PriorityQueue<>(
            (a, b) -> map.get(b) - map.get(a)
        );

        pq.addAll(map.keySet());

        int[] res = new int[k];

        for(int i = 0; i < k; i++){
            res[i] = pq.poll();
        }

        return res;
    }
}
```

⏱️ Complexity

- Time: O(n log n)

🎯 Final Summary

Problem	Concept
Majority Element	Boyer-Moore
Intersection	HashSet
Longest Substring	Sliding Window
Group Anagrams	Sorting + HashMap
Top K Frequent	Heap + HashMap

🔥 Interview Insight

👉 Agar:

- Frequency → HashMap
- Unique check → HashSet
- Window → Sliding Window
- Top elements → Heap